LAPORAN AKHIR PRAKTIKUM
PEMROGRAMAN BERBASIS OBJEK

Disusun Oleh:

Nofri Ilham

2511531013

Dosen Pengampu:

Nurfiah, S.ST, M.Kom.

Asisten Praktikum:

Ezza Andini



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

A. Pertemuan 1

Praktikum 1 membuat sistem perbankan mini berbasis CLI. Class Rekening memiliki atribut nomorRekening, namaPemilik, dan saldo, serta method setorTunai, tariktunai, dan cekInformasi sebagai Behavior. Constructor Rekening digunakan untuk inisialisasi atribut dan menampilkan pesan bahwa rekening berhasil dibuat.

Pada Main, ArrayList daftarRekening digunakan untuk menyimpan semua rekening. Variabel akunAktif bernilai null selama objek belum diinisialisasi, sehingga setiap menu terlebih dahulu memeriksa akunAktif dan menampilkan pesan error jika belum ada rekening. Behavior rekening dipanggil melalui akunAktif, misalnya akunAktif.setorTunai(setor). Perintah input.nextLine() setelah input.nextInt() digunakan untuk membersihkan buffer enter.

Aturan yang diterapkan: nominal setor harus lebih dari 0, sedangkan tarik tunai minimal 10000 dan tidak boleh melebihi saldo. Menu yang tersedia adalah Buka Rekening Baru, Setor Tunai, Tarik Tunai, Cek Informasi Rekening, Ganti Akun, dan Keluar. Tampilan program saat dijalankan adalah sebagai berikut.

```
=== SISTEM PERBANKAN MINI===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih Menu:
1
Masukkan No Rekening:
1234
Masukkan Nama Pemilik:
Nofri
Masukkan Saldo Awal:
1234567
 Rekening atas nama Nofri Berhasil dibuat dengan saldo Rp 1234567.0
Rekening berhasil ditambahkan.

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih Menu:
```

B. Pertemuan 2

Praktikum 2 mengimplementasikan asosiasi (1-to-many), yaitu satu Rekening memiliki banyak Transaksi. Class Transaksi memiliki atribut idTransaksi, jenis, dan nominal, serta method cetakDetail. Riwayat disimpan pada Rekening dalam bentuk ArrayList<Transaksi> riwayatTransaksi. ArrayList wajib diinisialisasi di dalam constructor agar tidak terjadi NullPointerException.

Riwayat direkam dengan membuat objek Transaksi di dalam method setorTunai (jenis Kredit) dan tariktunai (jenis Debit), dengan ID dari System.currentTimeMillis().

```java
public void setorTunai(double nominal) {
    if (nominal > 0) {
        saldo += nominal;
        // Merekam riwayat (pembuatan objek Transaksi di dalam method)
        String idTrx = "TRX-S-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
        riwayatTransaksi.add(trxBaru);
        System.out.println(" Setor tunai Rp " + nominal
            + " Berhasil. Saldo saat ini: Rp " + saldo);
    } else {
        System.out.println(" Gagal: Nominal setor harus lebih dari 0! ");
    }
}
```

Menu baru yang ditambahkan adalah Cetak Mutasi (cetakMutasi), yang menampilkan seluruh transaksi atau pesan bahwa belum ada transaksi pada rekening ini, dan Cek Transaksi (cektransaksi), yang hanya menampilkan riwayat jika jumlah transaksi lebih dari 3. Hasilnya seperti pada gambar berikut.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6.Cetak Mutasi
7.Cek Transaksi
0. Keluar
Pilih Menu:
6
ID: TRX-S-1792855827944 | Jenis: Kredit | Nominal: Rp500000.0
ID: TRX-S-1792855833994 | Jenis: Kredit | Nominal: Rp500000.0
ID: TRX-S-1792855840637 | Jenis: Kredit | Nominal: Rp100000.0
ID: TRX-T-1792855844874 | Jenis: Debit | Nominal: Rp150000.0
ID: TRX-T-1792855852595 | Jenis: Debit | Nominal: Rp200000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6.Cetak Mutasi
7.Cek Transaksi
0. Keluar
Pilih Menu:
```

C. Pertemuan 3

Praktikum 3 mengimplementasikan enkapsulasi. Atribut pada Rekening dan Transaksi dikunci dengan private. Getter hanya disediakan untuk atribut yang diizinkan dibaca publik, yaitu getNomorRekening dan getNamaPemilik pada Rekening, serta getIdTransaksi, getJenis, dan getNominal pada Transaksi. Class Transaksi hanya menyediakan Getter (Read-Only).

Pada Rekening ditambahkan atribut pin sebagai data sensitif. Constructor dimodifikasi untuk menerima PIN awal dan melakukan validasi: jika PIN bukan 6 digit, digunakan PIN default 123456. Di Main, PIN juga divalidasi dengan regex agar harus terdiri dari 6 angka. Method otentikasi merupakan validasi internal untuk memeriksa PIN yang dimasukkan, dan dipakai pada menu Tarik Tunai dan Cetak Mutasi. Jika PIN salah, program menampilkan pesan Akses Ditolak.



```
// Mengunci atribut dengan 'private'
private String nomorRekening;
private String namaPemilik;
private double saldo;
private String pin;   // Data Sensitif

// Getter untuk atribut yang diizinkan dibaca publik
public String getNomorRekening() { return nomorRekening; }
public String getNamaPemilik() { return namaPemilik; }

// Method Otentikasi Internal (Validasi enkapsulasi)
public boolean otentikasi(String inputPin) {
    return this.pin.equals(inputPin);
}
```

Mengapa tidak dibuat method setSaldo() dan getPin()?

Method setSaldo tidak diperlukan karena perubahan saldo sudah dilakukan melalui setorTunai dan tariktunai yang memiliki validasi. PIN adalah

data sensitif, sehingga tidak disediakan getPin agar class lain tidak dapat mengetahui PIN. Pemeriksaan PIN cukup dilakukan melalui method otentikasi.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6.Cetak Mutasi
7.Cek Transaksi
0. Keluar
Pilih Menu:
1
Masukkan No Rekening:
12345
Masukkan Nama Pemilik:
Nofri
Masukkan PIN(Harus 6 Digit)
12345
Pin harus terdiri dari 6 angka!
Masukkan PIN(Harus 6 Digit)
123456
Masukkan Saldo Awal:
10000000
 Rekening atas nama Nofri Berhasil dibuat
Rekening berhasil ditambahkan.
```

D. Pertemuan 4

Praktikum 4 mengimplementasikan inheritance. Rekening menjadi Superclass, sedangkan RekeningTabungan dan RekeningGiro menjadi Subclass. Atribut saldo dan riwayatTransaksi diubah menjadi protected agar Subclass bisa mengaksesnya langsung, sedangkan nomorRekening, namaPemilik, dan pin tetap private.

```
// Gunakan protected agar Subclass bisa mengaksesnya langsung
protected double saldo;
protected ArrayList<Transaksi> riwayatTransaksi;
```

Pada Subclass, constructor memanggil constructor kelas induk dengan super(), yang wajib berada di baris pertama. RekeningTabungan memiliki atribut spesifik sukuBunga dan method tambahBungaAkhirBulan, yang menghitung bunga dari saldo, menambahkannya ke saldo, dan mencatatnya sebagai transaksi berjenis Bunga.



```
public class RekeningTabungan extends Rekening {
    // Atribut spesifik yang hanya dimiliki oleh Tabungan
    private double sukuBunga;

    public RekeningTabungan(String nomor, String nama,
            double saldoAwal, String pinAwal, double sukuBunga) {
        // super() memanggil constructor kelas induk (Rekening),
        // WAJIB berada di baris pertama
        super(nomor, nama, saldoAwal, pinAwal);
        this.sukuBunga = sukuBunga;
    }

    public void tambahBungaAkhirBulan() {
        double nominalBunga = saldo * (sukuBunga / 100);
        saldo += nominalBunga;
        String idTrx = "TRX-B-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi(idTrx, "Bunga", nominalBunga));
    }
}
```

Mengapa class RekeningTabungan bisa mengakses saldo secara langsung?

Karena saldo dideklarasikan protected di class Rekening, sehingga dapat diakses oleh Subclass seperti RekeningTabungan, tetapi tidak dapat diakses oleh class lain yang bukan turunannya.

RekeningGiro memiliki atribut spesifik batasOverdraft dan method getBatasOverdraft. Pada Main, pengguna memilih produk saat membuka rekening, dan menu Simulasi Akhir Bulan menggunakan instanceof dengan casting ke RekeningTabungan. Jika akun aktif bukan Tabungan, program menampilkan pesan gagal karena fitur bunga hanya berlaku untuk Rekening Tabungan.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6.Cetak Mutasi
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih Menu:
1
Masukkan No Rekening:
1234
Masukkan Nama Pemilik:
Nofri
Masukkan PIN (6 digit):
123456
Masukkan Saldo Awal:
10000000

Pilih Produk:
1. Tabungan Umum
2. Giro Bisnis
Pilih Produk:
1
Masukkan Suku Bunga (%):
5
 Rekening atas nama Nofri Berhasil dibuat
Rekening berhasil ditambahkan.

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6.Cetak Mutasi
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih Menu:
7
Bunga 5.0% berhasil ditambahkan: Rp500000.0
```

E. Pertemuan 5

Praktikum 5 mengimplementasikan polimorfisme dan pengikatan dinamis (Dynamic Method Dispatch), yaitu JVM menentukan method mana yang dipanggil pada saat program berjalan sesuai tipe objek aslinya. Pada praktikum ini dilakukan Method Overriding agar RekeningGiro mengizinkan saldo minus hingga batas overdraft-nya, serta dibuat method yang menerima argumen polimorfik.

Langkah 1: Refaktor method tarikTunai pada Rekening

Agar fungsi penarikan dapat digunakan sebagai bagian dari fitur transfer, tipe kembalian tarikTunai diubah dari void menjadi boolean, yaitu true jika transaksi sukses dan false jika gagal.



```java
// Mengubah void menjadi boolean
public boolean tarikTunai(double nominal) {
    if (nominal <= 0) {
        System.out.println("Gagal: Nominal tarik harus lebih dari 0!");
        return false;
    } else if (nominal > saldo) {
        System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
        return false;
    } else {
        saldo -= nominal;
        String idTrx = "TRX-T-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi(idTrx, "Tarik Tunai", nominal));
        System.out.println("Tarik tunai Rp" + nominal + " berhasil. Sisa saldo: Rp" + saldo);
        return true; // Transaksi sukses
    }
}
```

Code diatas merupakan perubahan pada class Rekening. Dengan nilai kembalian boolean, pemanggil method dapat mengetahui apakah penarikan berhasil atau tidak.

Langkah 2: Overriding pada RekeningGiro

Pada RekeningGiro, method tarikTunai ditimpa dengan anotasi @Override agar menggunakan aturan batasOverdraft.

Code diatas adalah overriding method tarikTunai pada class RekeningGiro. Penarikan diizinkan selama nominal tidak melebihi saldo ditambah batasOverdraft, sehingga saldo dapat bernilai negatif. Anotasi @Override memastikan compiler memeriksa bahwa method ini benar-benar menimpa method pada class induk.

Langkah 3: Uji coba pengikatan dinamis

Kode pada menu 3 (Tarik Tunai) di Main tidak diubah sama sekali. Pada akun Giro dengan saldo Rp100.000 dan batas overdraft Rp500.000, dilakukan tarik tunai Rp400.000. Karena objeknya adalah RekeningGiro, Java otomatis memanggil tarikTunai milik RekeningGiro sehingga penarikan berhasil dan

saldo menjadi minus Rp300.000. Jika pemanggilan yang sama dilakukan pada Rekening biasa, penarikan ditolak karena saldo tidak mencukupi.

```
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6.Cetak Mutasi
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih Menu:
1
Masukkan No Rekening:
1234
Masukkan Nama Pemilik:
Nofri
Masukkan PIN (6 digit):
123456
Masukkan Saldo Awal:
100000

Pilih Produk:
1. Tabungan Umum
2. Giro Bisnis
Pilih Produk:
2
Masukkan Batas Overdraft:
500000
 Rekening atas nama Nofri Berhasil dibuat
Rekening berhasil ditambahkan.

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6.Cetak Mutasi
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih Menu:
3
Masukkan PIN
123456
Masukkan Nominal Tarik tunai
400000
Tarik tunai Giro Rp400000.0 berhasil. Saldo saat ini: Rp-300000.0
```

Tugas: Fitur Transfer Antar Rekening

Pada class Rekening ditambahkan method transfer dengan parameter polimorfik Rekening tujuan, sehingga dapat menerima RekeningTabungan

maupun RekeningGiro. Logikanya: otentikasi PIN pengirim dan batalkan jika salah, panggil this.tarikTunai(nominal), lalu jika penarikan berhasil setor ke rekening tujuan dengan tujuan.setorTunai(nominal). Jika pengirimnya Giro, aturan overdraft otomatis digunakan.



```
public void transfer(Rekening tujuan, double nominal, String pinInput) {
    // 1. Autentikasi PIN
    if (!otentikasi(pinInput)) {
        System.out.println("Transfer gagal: PIN salah!");
        return;
    }

    // 2. Tarik saldo dari rekening pengirim
    boolean berhasil = this.tarikTunai(nominal);

    // 3. Jika penarikan berhasil, setor ke rekening tujuan
    if (berhasil) {
        tujuan.setorTunai(nominal);
        System.out.println("Transfer berhasil!");
    } else {
        System.out.println("Transfer gagal!");
    }
}
```

Selanjutnya pada Main ditambahkan menu 8. Transfer ke Rekening Lain. Program meminta nomor rekening tujuan, mencarinya di daftarRekening, lalu meminta nominal transfer dan PIN, kemudian memanggil akunAktif.transfer(akunTujuan, nominal, pin).



```java
case 8:
    System.out.print("Masukkan Nomor Rekening Tujuan: ");
    String nomorTujuan = input.nextLine();

    Rekening akunTujuan = null;

    // Cari rekening tujuan
    for (Rekening rekening : daftarRekening) {
        if (rekening.getNomorRekening().equals(nomorTujuan)) {
            akunTujuan = rekening;
            break;
        }
    }
    if (akunTujuan == null) {
        System.out.println("Rekening tujuan tidak ditemukan!");
        break;
    }
    if (akunTujuan == akunAktif) {
        System.out.println("Tidak dapat mentransfer ke rekening sendiri!");
        break;
    }
    System.out.print("Masukkan nominal transfer: Rp");
    double nominal = input.nextDouble();
    input.nextLine();
    System.out.print("Masukkan PIN Anda: ");
    String pin1 = input.nextLine();
    akunAktif.transfer(akunTujuan, nominal, pin1);
    break;
```

Code diatas mencari rekening tujuan dan menolak transfer jika rekening tidak ditemukan atau sama dengan akun aktif. Saat dijalankan, transfer dilakukan dari akun Giro “Nofri” (saldo -Rp300.000 setelah uji coba, batas overdraft Rp500.000) ke akun Tabungan “Ilham” (saldo Rp200.000) sebesar Rp150.000. Penarikan masih dalam batas overdraft sehingga berhasil, saldo “Nofri” menjadi -Rp450.000 dan saldo “Ilham” menjadi Rp350.000.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6.Cetak Mutasi
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih Menu:
5
Masukkan No Rekening:
1
Berhasil mengganti akun.
Akun aktif: Nofri

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6.Cetak Mutasi
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih Menu:
8
Masukkan Nomor Rekening Tujuan: 123
Masukkan nominal transfer: Rp150000
Masukkan PIN Anda: 123456
Tarik tunai Giro Rp150000.0 berhasil. Saldo saat ini: Rp-450000.0
 Setor tunai Rp 150000.0 Berhasil. Saldo saat ini: Rp 350000.0
Transfer berhasil!
```

F. LINK GITHUB DAN REPOSITORY

Kode program dapat dilihat pada repository GitHub melalui link di bawah ini:

https://github.com/nofriilham/PrakPBO_C_2511531013/tree/Pertemuan5

Video penjelasan berupa video YouTube yang dapat diakses melalui link di bawah ini:

https://youtu.be/36bKDNOFVmE?si=8B-0SdRm99XNUwoc